N83: a measured Bernstein–Lange arm against the index calculus

Status (2026-10-15): the rho step implementation is set, and the run starts on push. `rho_reference.value` in `protocol.json` is `rT770`: A's tuned `StrongRhoG<Gf2_128 fold>`, with 64 lanes and DP 6. Item 2 chose it: the reconciliation, run 38617911080 on branch `claude/n83-reconcile`, found it to be the fastest correct reference. Nothing else in the protocol changed. No n=83 result exists yet, and this directory makes no performance claim until the run lands. The sections below describe the design as it was frozen, while the value was still open.

Question

The crypto ledger, `research/notes/index-calculus/RESEARCH_IC_BOUNDARY_LEDGER.md` §23, reports the index calculus's online cost beside a *model*. The model is Bernstein and Lange's precomputation rho, given the index calculus's setup as its precomputation budget. B's Tier 1 run (`experiments/koblitz-single-target-n83-tier1-20261014/`, run 38523707244) carried that column at n=83: the model put the index calculus 33.5× slower online than BL, given the same 882 s setup.

This experiment replaces the model with a measurement on `EC1N83Ce1h35c7ced6b497` (a=1, n=83, r = 8569786107849059 ≈ 2^52.93, 166 signed-Frobenius automorphisms):

- **Precomputation.** A precomputation rho on the $\langle \tau, -1 \rangle$ classes builds a table of T distinguished points. Its walks use the same jump function, canonicalization, distinguished-point rule, lanes and batched step as the online walk.
- **Budget.** T is not chosen in advance. The precomputation runs until its cost, in wall milliseconds on the same pinned CPU of the same runner, equals the index calculus's setup in the same job: base + relations + log table, P_IC.
- **Online.** From each target, the walk runs until it hits a stored distinguished point. It then solves and verifies [d]G = Q.
- **Targets and comparison.** The same Q0 (4 blocks) and B's 16 fresh public points are used. The arms are:
  - the BL online cost;
  - B's IC producer with the cached log table;
  - the single-target rho;
  - the cold cost of every arm.

Did a BL implementation exist? (step 1)

A search of crypto `origin/main` (bernstein, lange, precomput, distinguished table) found two.

- **`examples/koblitz_rho_batch_ks.rs` (and its `_matched_arith` twin).** It has `KIC_RHO_PRECOMPUTE_WALKS`: G-only walks whose distinguished points enter the batch table as `PRECOMPUTED` trails. It is the packed `u64` backend: one walk at a time, n ≤ 63, with no lockstep lanes. It is not the strong signed-Frobenius walk, and it cannot build n=83.
- **`examples/prime_ecdlp_breakthrough_probe.rs`.** It is for prime fields only.

The strong walks have no precomputation. These are `koblitz_rho_batch_ks_strong_online.rs` (rung 3) and the library's `StrongRhoG` (`koblitz_strong_rho.rs`). Their own header says "no Bernstein–Lange precomputation".

So this experiment implements BL on the strong walks. It reuses the old design: a precomputed entry is a trail with b = 0, so the walk's existing collision code solves a hit unchanged.

The open parameter: which rho step implementation

Item 2 compares three implementations. Each is a protocol value here, and each is implemented and tested:

value	reference single-target rho (built unchanged from crypto 770f6a69)	BL arm (generated from that same file)
`rB770`	`koblitz_rho_batch_ks_strong_online` rung 3, 32 lanes, DP 4 (B's)	`bl_rung3`: rung 3's wide path with a table preload and a precompute mode
`rA770`	`n83_strong_point_eval`: `StrongRhoG<WideGf2,u64>`, 32 lanes, DP 4 (A's)	`bl_sg --field wide`: the library walk with `solve_with_table` and `bl_precompute`
`rT770`	`n83_rho_optimization_eval`: `StrongRhoG<Gf2_128 fold>`, 64 lanes, DP 6 (A's tuned)	`bl_sg --field fold`

How each BL arm is built. `restore_sources.py` makes every BL arm from the pinned file itself. It uses exact textual edits, each asserted to match exactly once, and every input and output is digest-bound in `protocol.json`:

- `sg_bl_module.rs` is the library walk with two functions appended.
  - `solve_with_table` is the library's `solve` with four substitutions. The distinguished-point table is passed in instead of created empty, and the successful collision reports whether its stored partner had b = 0.
  - `bl_precompute` contains `solve`'s step block.
- `bl_rung3.rs` is B's example with three changes to its wide `run`. The jump seed may come from `KIC_BL_JUMP_SEED`. A table may be preloaded before the target is built. One extra JSON line is printed. A `KIC_BL_MODE` precompute and verify path is added. With no BL variable set, `run` is the frozen file's.

What is left to fill in. One line in `protocol.json`: set `"value": "rB770" | "rA770" | "rT770"` under `rho_reference`, and change `status` from `OPEN` to `SET`. Then commit and push to `claude/n83-bl-arm`. Nothing else depends on the choice:

- `bl_table.py` and `run_job.py` read the value;
- the sources digests already cover all three arms;
- the positive controls run for all three in every job.

Design

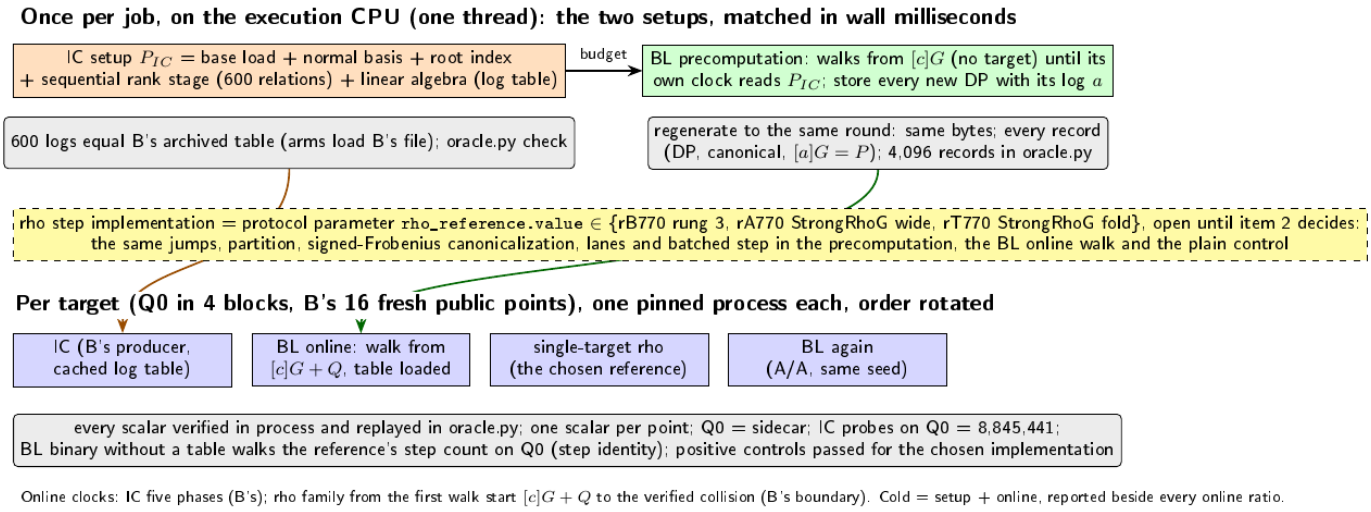


Figure 1: Design

Figure 1. Budget matching, the shared walk and the per-target arms (source, vector). Schematic.

- **Budget.** `ic_precompute.py` runs B's rank stage once, cold, on Q0, on the execution CPU with one thread. B's `precompute.py` does the same.
  - P_IC = `rank_stage + setup_total`: base load, normal basis and self-test, root index, the sequential rank stage, and linear algebra.
  - The produced factor-log table must equal B's archived one (`a771716f...`) in every field except `provenance.rank_stage_ms`. The file records its own rank-stage time, so its bytes cannot repeat. The IC arms load B's archived file itself.
  - Left out: the construction of the retained K600 base (2026-10-06). That makes BL's budget smaller than the IC's true setup, so the omission can only favour the IC.
- **Precomputation** (`bl_table.py precompute`). The precomputation:
  - uses the chosen implementation's walk, with distinguished-point bits dp = 6;
  - takes its jumps from the fixed seed 20261014 and its starts c·G from the seed 20261015;
  - stops at the first 256-round boundary at which its own clock reads at least P_IC;
  - writes `table.bin`: 40-byte records (x, y, a), sorted, with SHA-256 and BLAKE3 recorded.
- **Table checks** (`bl_table.py check`, before any online arm, untimed):
  - regeneration to the recorded round count must give the same bytes;
  - the producer checks every record: distinguished, canonical, [a]G = P;
  - the pure-Python `oracle.py` checks a seeded sample of 4,096 records.
- **Panel** (`run_job.py`). The targets run in this order: Q0 block 0, w00–w03, block 1, w04–w07, and so on.

  - Every target gets four processes: IC, BL, rho and BL again (the A/A). They run one at a time on one pinned CPU, in rotation k mod 4.
  - One more run, the step-identity control, comes last.
  - The walk seeds are B's: 831000–831003 on Q0 and 830000 + w on the fresh points. BL and rho share start streams; the jump tables differ (BL uses the table's).
- **Clocks.**
  - IC: B's five phases.
  - Every rho-family arm: B's boundary, from the first walk start [c]G+Q to the collision whose [d]G = Q check passed. The table load, curve, jumps and membership check are excluded and logged. A's boundary is kept per row.
- **Statistics** (protocol `statistics`).
  - **Primary.** On the 16 fresh points: mean IC online over mean BL online, with a 10,000-draw bootstrap over targets.
  - **Beside it:**
    * rho over BL;
    * rho over IC;
    * the Q0 blocks;
    * the cold costs (setup + online) of all three;
    * the single-target rho cost;
    * T, its α, and the model check.
- **Decision rule.** "BL online faster than IC" requires all of:
  - the lower bound of the interval is above 1;
  - every row is valid and verified;
  - every scalar replays in `oracle.py`;
  - the budget matched;
  - the table checks, the step-identity check, the IC work identity and the A/A step identity all pass;
  - the chosen implementation's positive controls passed.

  Otherwise the reading is `no_result`.

Sizing T for 4 vCPU / 16 GB / 6 h (step 2)

The matched budget fits, at full scale. The matched fraction is 1.0, and no scaling is needed. The precomputation takes about P_IC of one CPU. At B's figure that is 882 s: 835.5 s rank stage plus 46.7 s setup. Its table:

implementation, ns per step (source)	P (steps)	T at dp 6	table file	peak RAM (est.)	model online steps	model online ms
rB770, 706.7 (B's runner, measured)	1.25e9	1.95e7	0.78 GB	3.1 GB	43,400	30.7
rA770, 461 (A's host)	1.91e9	2.99e7	1.2 GB	5.8 GB	29,000	13.4
rT770, 250 (assumed) to 461	3.5e9 to 1.9e9	5.5e7 to 3.0e7	2.2 to 1.2 GB	7.0 to 5.8 GB	18,700 to 31,100	4.7 to 14.3

These are predictions, not results. The run measures its own P_IC and its own ns per step.

The job's budget (protocol `resources`):

stage	estimate
build and tests	7–20 min
positive controls	1–10 min
IC rank stage	15–35 min
BL precomputation	equal to the IC rank stage
table checks	10–40 min
panel (81 processes)	25–45 min
total	about 1.5–3.2 h, inside the 350-minute job

Memory. One process runs at a time, and the table checks run in sequence. The largest is the IC arm, 4.4–5.9 GB, or a BL process at the largest T, about 7 GB.

Why dp = 6, for every implementation. The online cost under the model is l/c + lanes·2^dp, with l = (r−1)/166 classes and c ≈ P covered points.

- dp 5 would halve the lane term, but needs up to 1.1e8 entries.
- dp 7 adds 2,000–4,000 steps, 5–20% of the online cost.
- At these α (0.04–0.07), the table holds nearly every walk's endpoint: duplicates are under 1%.

The model (heuristic estimate, not a measurement)

Plain rho: sqrt(πl/2) + lanes·2^dp steps.

BL online: sqrt(πl/2)·exp(z^2)·erfc(z) + lanes·2^dp, with z = c/sqrt(2l), where c is the number of trail points the precomputation covered.

- This is the first merge for a walk that meets the stored trails with probability c/l per step and its own past with probability t/l. Detection then costs about one walk length per lane.
- It equals l/c + lanes·2^dp for c $\gg$ sqrt(l), and plain rho at c = 0.
- Bernstein and Lange's tuned table selection reaches about 1.93·sqrt(l/T) at a precomputation of 1.21·sqrt(lT). That is the ledger's column. This arm stores every distinguished point it finds, with no selection, which the model above describes.

Controls and local validation (step 3 and step 5)

The container is shared and contended, so every clock below is a diagnostic. Receipts are in `smoke/`.

Positive control (counts only)

`control.py` covers every implementation, each on its exact n=83 code path:

- E_0 over GF(2^41) (r = 549756390943), for `rA770`. `WideGf2` accepts n=41, and its table equals the `Gf2` table byte for byte.
- E_0 over GF(2^69) (r = 254196239707 ≈ 2^37.9, 138 automorphisms), the smallest rung on the wide path, for all three.

The procedure is the same for each:

1. Build tables with a step-count stop at two sizes, and verify every record.
2. Solve 256 known-answer targets with the table.
3. Solve the same 256 targets without it (plain rho, same binary, same jumps, same DP rule).

Every scalar was recovered. The counts are deterministic: a second run gave identical rows.

control	P (steps)	T	plain rho mean [95%] / model	BL mean [95%] / model	table hits	BL speedup over plain: measured / model
n=41, rA770	2^19	34,692	96,123 [89,774, 102,395] / 103,133 (0.932)	11,257 [9,997, 12,568] / 12,383 (0.909)	254/256	8.54 / 8.33 (1.025)
n=41, rA770	2^21	139,283	same	3,325 [2,960, 3,714] / 3,544 (0.938)	256/256	28.91 / 29.10 (0.993)
n=69, rA770	2^17	8,828	51,688 [48,401, 54,892] / 54,302 (0.952)	11,109 [9,985, 12,260] / 12,763 (0.870)	238/256	4.65 / 4.25 (1.094)
n=69, rA770	2^18	17,700	same	5,647 [5,061, 6,260] / 6,988 (0.808)	253/256	9.15 / 7.77 (1.178)
n=69, rB770	2^17	8,738	49,820 [46,460, 53,245] / 54,302 (0.917)	11,833 [10,611, 13,098] / 12,828 (0.922)	235/256	4.21 / 4.23 (0.995)
n=69, rB770	2^18	17,410	same	5,972 [5,342, 6,631] / 7,016 (0.851)	248/256	8.34 / 7.74 (1.078)
n=69, rT770	2^17	8,825	52,071 [48,771, 55,275] / 54,814 (0.950)	11,530 [10,400, 12,686] / 13,308 (0.866)	240/256	4.52 / 4.12 (1.096)
n=69, rT770	2^18	17,694	same	5,999 [5,409, 6,612] / 7,516 (0.798)	254/256	8.68 / 7.29 (1.190)

Figure 2. Positive control, recorded step counts from `smoke/control/*/control.json`, 256 targets per point, with 95% bootstrap intervals over targets (source written by `figures/make_figures.py`, data, vector). Left: steps against the model. Right: the gate, BL's speedup over plain rho against the model's speedup.

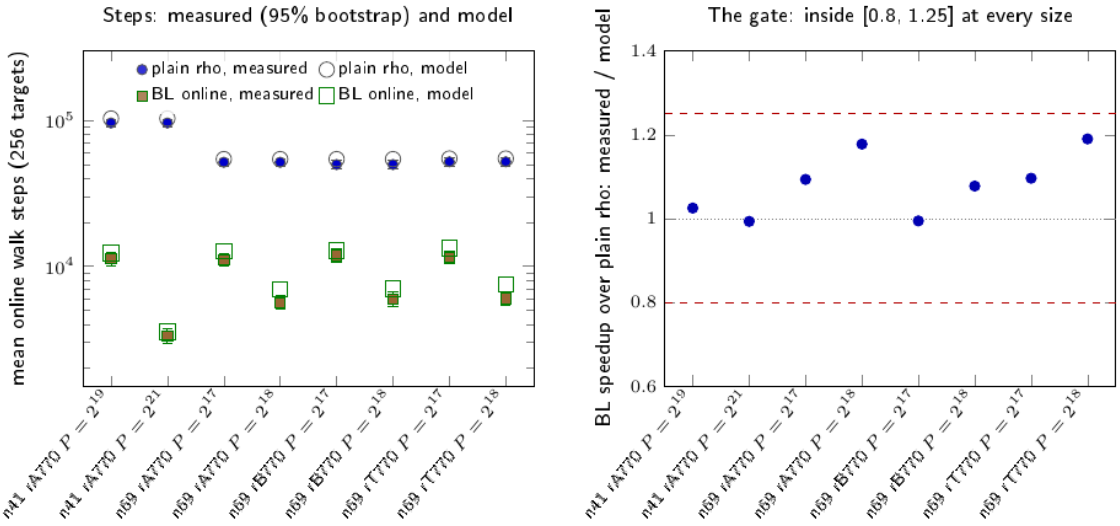


Figure 2: Positive control

Verdict. The positive control passes for all three implementations under the gate as amended. BL's speedup over plain rho matches the model at 0.99–1.19 of it, inside [0.8, 1.25].

Amendment A1, made before the protocol was committed and disclosed here. The first gate also put the absolute BL step count inside [0.8, 1.25] of the model. In this dry run it read 0.80–0.94, and one of eight points fell just outside: n=69, rT770, 2^18, at 0.798. Plain rho also read 0.92–0.95 of its model. So the random-mapping model overstates both arms by a similar 5–10% at these sizes, and the speedup ratio cancels that.

The gate is now the speedup ratio, which is the stated criterion. The band is unchanged. The absolute ratios are still reported. Both versions are in `protocol.json` (`amendments`).

Step identity (exact)

Run without a table and with the reference's own jumps and DP rule, each BL binary walks Q0 in exactly the archived step counts:

BL binary	seed	steps	archived reference
`bl_rung3`	831000	12,804,216	B's rung 3, run 38523707244
`bl_sg --field wide`	202610063	5,353,957	A's StrongRhoG
`bl_sg --field fold`, 64 lanes, DP 6	831000	12,592,070	reconciliation smoke, rT770

Each recovered 3141592653589793. `cargo test` adds a further check: with an empty table, the generated `solve_with_table` returns the library `solve`'s scalar, steps, walks and charges.

Reduced n=83 smoke

A 60 s budget replaced P_IC. Each implementation ran: precomputation, the three table checks, then Q0 block 0 and w00 with all four arms, plus the step-identity run. Receipts are in `smoke/n83/`.

impl	T (60 s)	precompute steps	ns/step (contended)	table checks	BL steps Q0 / w00	rho steps Q0 / w00	plain BL binary on Q0	valid runs
rB770	1,512,878	95,357,424	629	PASS	382,656 / 446,560	12,804,216 / 9,447,525	12,804,216	9/9
rA770	1,892,168	119,242,797	503	PASS	347,115 / 416,546	9,413,926 / 8,062,650	9,413,926	9/9
rT770	2,917,989	183,941,223	326	PASS	305,331 / 363,884	12,592,070 / 6,487,138	12,592,070	9/9

- **Exact identities.** Every scalar was verified in process and replayed in `oracle.py`; Q0's equals the sidecar. On every target, BL and BL' took identical steps. Every BL solve hit the precomputed table. The IC took B's 8,845,441 probes on Q0 and 14,455,639 on w00, the reconciliation's count.

- **Step identity.** The plain BL binary's Q0 count equals its reference rho's, row for row.
- **IC precomputation.** The IC precompute (`ic_precompute.py`) ran here once, for 22 minutes on a contended CPU. Its 600 logs equal B's archived table.
- **No timing reading.** These receipts contain wall times, but they are contended diagnostics on a reduced table, and no ratio from them is reported.

Build, tests, lint

- `cargo build --release --locked` passes. It builds all six binaries: `ic_online_logcache_r770`, `rho_rung3_r770`, `rho_adapter_r770`, `rho_tuned_r770`, `bl_rung3` and `bl_sg`.
- `cargo test --release --locked --bins` passes: 27 tests (`smoke/cargo-test.txt`). They include `bl_sg`'s two tests: the empty-table solve equals the library solve, and the precomputation is exact, reproducible and used online.
- `actionlint` 1.7.12, with shellcheck and pyflakes, passes on `.github/workflows/n83-bl-arm.yml`. `pyflakes` passes on every script.
- All of this used local rustc 1.97.0. The workflow pins rustc 1.99.0, as B and the reconciliation did.

Claim classification (controlled-experiment skill)

- **Exact computation.**
  - Every recovered scalar.
  - The table's records.
  - Step identity, and the IC probe identity.
- **Comparative.** The n=83 online ratios, read by the frozen rule and quoted with their intervals and with the cold costs. None exists yet.
- **Heuristic estimate.**
  - The model.
  - The sizing table.
  - The BL paper's 1.93·1.21 constants.
- **Out of scope.**
  - Any statement about ECC2K-130, E_0 over GF(2^83), or the speed of either method in general.
  - Any wall-clock figure from this container.

Workflow

`.github/workflows/n83-bl-arm.yml` runs on a push to `claude/n83-bl-arm` or by `workflow_dispatch`. Its path filter excludes, from the start:

- `run_github_*/**` (receipts);
- `smoke/**`;
- `README.md` and `README.pdf`;
- `figures/**` (including the figure script);
- `result*`.

The workflow:

1. Restores and checks the sources, then builds and tests with rustc 1.99.0.
2. Runs the positive controls.
3. **Stops if `rho_reference.value` is `null`.**
4. Runs the IC rank stage, then the BL precomputation at P_IC.
5. Runs the table checks, then the panel.
6. Runs `verify_receipt.py`.

It uploads two artifacts: the receipt, and `table.bin` on its own, uncompressed.

Receipt

The receipt (`$RUNNER_TEMP/n83-bl-arm/`) holds:

- `allocation.json`, `host-probe.json`, `rustc.txt`, `cargo-test.txt`, `binaries.sha256`, `frozen.sha256`, `schedule.jsonl`;
- `control/`: `summary.json` and, per control, `control.json` and `rows.jsonl`;
- `ic_precompute/`: `precompute_receipt.json` (P_IC), the factor-log table and its oracle check;

- `bl/`: `precompute_receipt.json`, `table_checks.json` and the regeneration's stdout. `table.bin` is uploaded as its own artifact.
- `panel/`: `runner_start.json`, `runs.jsonl` and `runs/*.stdout.txt` (the base64 raw producer records), `runner_end.json`;
- `result.json` and `result_table.md`.

Files

file	role
`protocol.json`	the frozen design; `rho_reference.value` open
`restore_sources.py`	restores crypto 770f6a69 and generates the BL arms (digest-checked)
`bl_src/`	the BL code: `sg_bl_impl.rs`, `bl_sg.rs`, `rung3_bl_impl.rs`, `bl_table_io.rs`
`Cargo.toml`, `Cargo.lock`	the build
`bl_tools.py`	command lines and the model
`control.py`	the positive control
`ic_precompute.py`	B's rank stage, giving P_IC
`bl_table.py`	the BL precomputation and the table checks
`arm.py`	one arm on one point
`run_job.py`	the panel
`verify_receipt.py`	the audit and the decision rule
`figures/`	figures, sources and the figure script
`smoke/`	local validation receipts

Reproduce

```
git clone https://github.com/aburan28/crypto /path/crypto
python3 restore_sources.py --crypto-repo /path/crypto
cargo build --release --locked && cargo test --release --locked --bins
python3 control.py --bin-dir $CARGO_TARGET_DIR/release --out R/control
python3 ic_precompute.py run --binary $CARGO_TARGET_DIR/release/ic_online_logcache_r770 --cpu 0
↪  --out R/ic_precompute
python3 bl_table.py precompute --bin-dir $CARGO_TARGET_DIR/release --cpu 0 --ic-receipt
↪  R/ic_precompute/precompute_receipt.json --out R/bl
python3 bl_table.py check --bin-dir $CARGO_TARGET_DIR/release --receipt-dir R/bl --regen-cpu 1
↪  --verify-cpus 0,1,2,3
python3 run_job.py --bin-dir $CARGO_TARGET_DIR/release --log-table
↪  ../koblitz-single-target-n83-tier1-20261014/tier1_github_38523707244/precompute/factor_logs_n83_K600.jso
↪  \
  --bl-receipt-dir R/bl --receipt-dir R/panel --cpu 0
python3 verify_receipt.py R
```

Limits

- **Not yet run at n=83.** The n=83 smoke was reduced: a 60 s budget and two targets. It ran on a contended container.
- **Tier 1 at best.** One runner and one pinned CPU on a shared cloud VM. Exclusivity is not certified.
- **The model is a heuristic.** In the dry run it overstated step counts by 5–20% at small sizes.
- **No Bernstein–Lange table selection.** BL's tuned selection, which keeps the most useful points, would lower BL's online cost further at the same budget. This arm measures the plain variant.
- **The IC budget leaves out base construction.** That favours the IC.
- **Not the same binaries as item 2.** The reference binaries are built from the same files, lock and toolchain, but in this crate, so binary identity with item 2's builds is not claimed. Work identity is checked instead, through the step and probe counts.
- **Canonical graphs checked: no change.** These were checked and left unchanged, because no n=83 BL measurement exists yet:
  - this repository's `experiments/koblitz-single-target-n83-tier1-20261014/figures/`;
  - the crypto ledger §23 and the scoreboard.

  The ledger row, the scoreboard and any `benchmarks/` entry belong to the PR that lands the run.